Why AI for QA?

Developers using AI write more code, faster → QA without acceleration or automation can become a bottleneck.

Why QA for AI?

AI generation results can be unstable, and generation pipelines also need testing. Quality assessment approaches are applicable in any field.

AI makes mistakes without context

Without context:

- Uses contradictory requirements
- Generates fewer tests (~30-50% less)
- May miss checks, even for boundary values
- Creates numerous explanatory .md files (e.g., Haiku)

With context:

- Easy to execute several small pre-setup operations
- Provides better requirements coverage
- Generates API tests according to the specified structure and rules

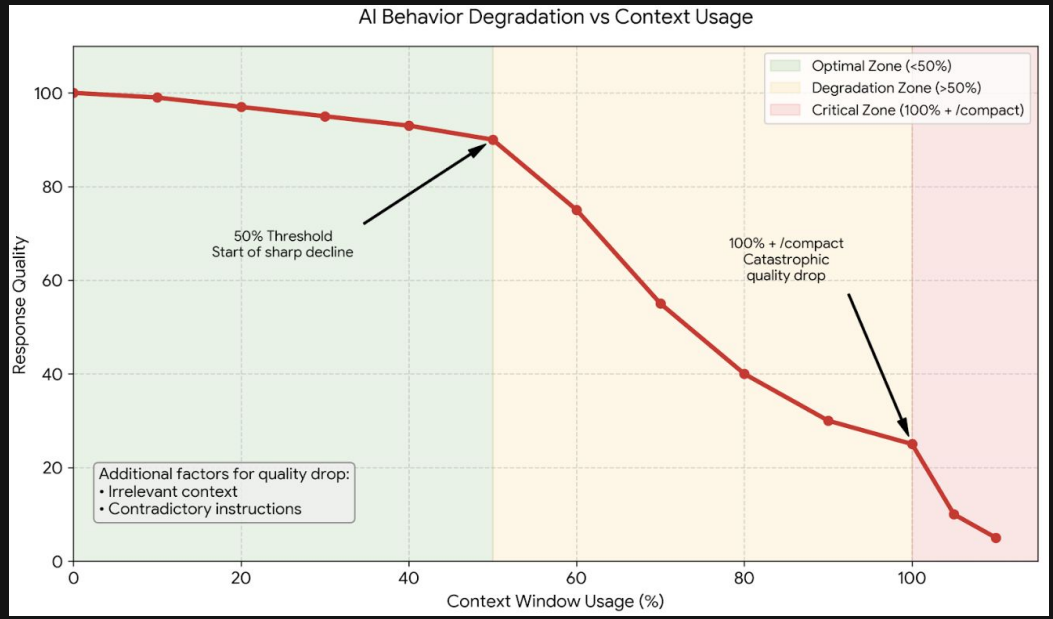
Context overload is bad

```
/context
└ Context Usage
   ⛁ ⛁ ⛀ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁   claude-sonnet-4-6 · 34k/200k tokens (17%)
   ⛁ ⛁ ⛁ ⛀ ⛁ ⛁ ⛁ ⛁ ⛁ ⛶
   ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   Estimated usage by category
   ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ System prompt: 5.1k tokens (2.6%)
   ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ System tools: 16.2k tokens (8.1%)
   ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ MCP tools: 2k tokens (1.0%)
   ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ Memory files: 1.7k tokens (0.9%)
   ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ Skills: 757 tokens (0.4%)
   ⛶ ⛶ ⛶ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒   ⛁ Messages: 9.6k tokens (4.8%)
   ⛒ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒   ⛶ Free space: 132k (65.8%)
                        ⛒ Autocompact buffer: 33k tokens (16.5%)
```

* 8,044 lines in *.md files

Behavior degradation

- When exceeding 50% of the context window
- With irrelevant context
- With contradictory instructions
- After reaching 100% of the context window and using 'compact'



Effective context management:

- Separate, small instructions
- Clearing context (/clear)
- Saving artifacts before closing the session
- Outputting to a .md or code file is cheaper (more efficient) than to the chat
- Operation decomposition
- Updating Claude.md / SKILL.md to avoid repeating mistakes

How to choose which layer to add instructions to

CLAUDE.md (rules.md) < 300-500 lines (Readme for AI)

- Stack, Architecture, Critical Rules

agent.md < 300-500 lines

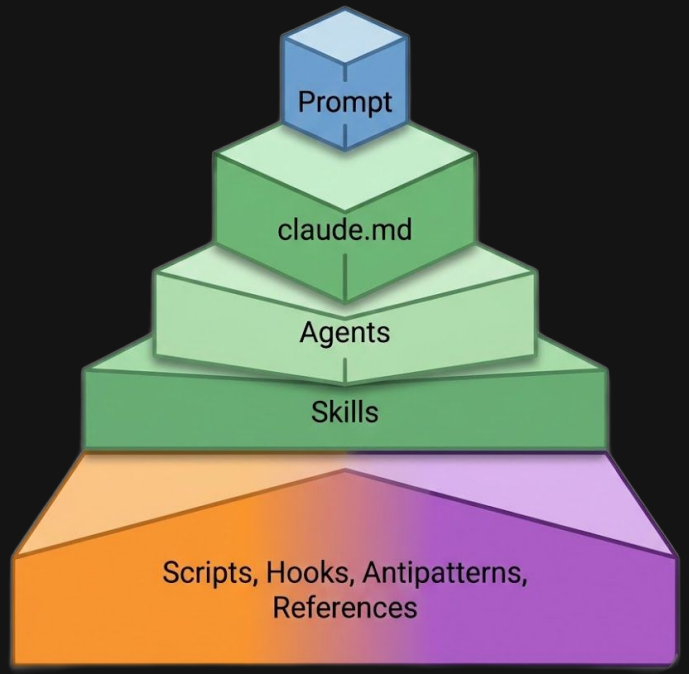
- Mindset + Skill Map + Links to Patterns and Antipatterns
- Definition of Done: What is considered a completed task (code, test, report)

SKILL.md < 500 lines or more - step-by-step guides:

- Checking requirements
- Writing manual test cases
- Writing API tests

When the limit is exceeded:

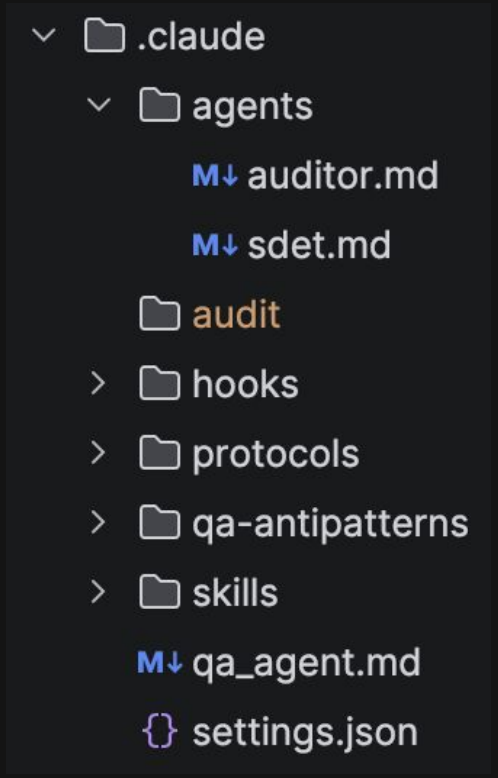
- /references
- /antipatterns



Everything can be automated, but it shouldn't be

- Checking document consistency
- Checking the correct linkage of multiple documents
- Automated suggestions for document correction and moving specific parts to lower layers when volume is exceeded
- Automatically updating documentation "for itself" (what's in the setup and how it works)
- Loop: run autotests, fix errors (max X attempts)

Keep agent architecture lean

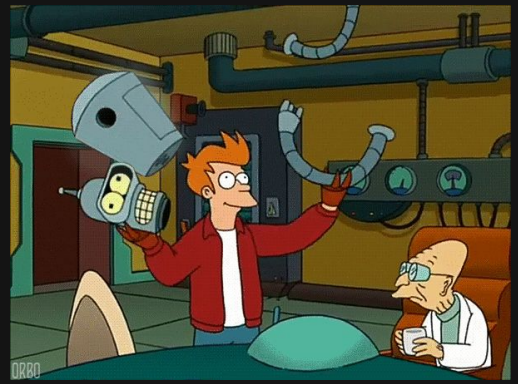


- Orchestrator, manager, reviewer
- Analyst, reviewer
- Writer

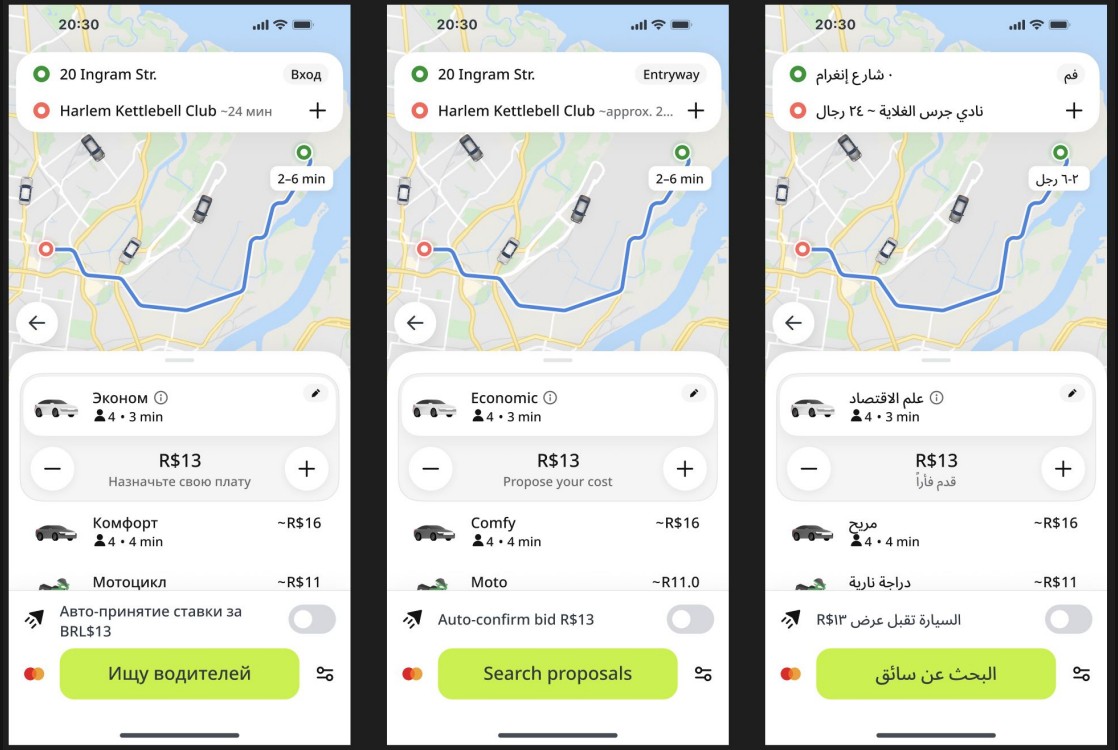
Onboarding to a new repository

- /init-project: generate Claude.md for the backend repo
- /repo-scout: analyze the target repository
- /test-cases: create a list of tests
- Start the API tests repo by creating:
    - /init-project: Claude.md
    - /init-agent: agent.md
    - /init-skill: SKILL.md
- /api-tests: generate API tests (use /test-cases output)

*Human in the loop (HITL)



Visual and linguistic analysis



Hands-on Practice

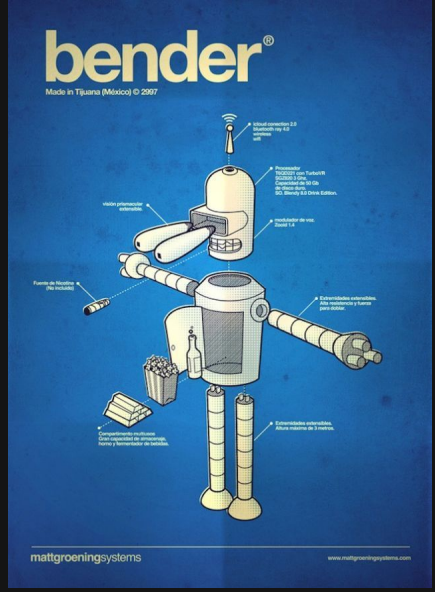
Clone the repository:
https://github.com/vlad-ryzhkov/ai-context-engineering-for-qa

Ensure you have access to an AI assistant

Main demonstration stack: IntelliJ IDEA + Claude Code CLI

Alternatives (see README):

- IDE: Cursor, VS Code, OpenCode
- AI: GitHub Copilot, Gemini, any AI chat



Minimalistic Stack

Component	Purpose
.claude/settings.json	Permissions, hooks, plugins, MCP config
kotlin-lsp plugin	Live IDE diagnostics inside Claude
claude-plugins-official	Official Anthropic plugin registry
context7 MCP	Up-to-date library docs on demand
sequential-thinking MCP	Structured step-by-step reasoning

Ideas for independent practice

- MCP Playwright + Jira to run website analysis and create bug reports in Jira (a 30-60 minute adventure)

- Agent/SKILL for code review

- Package your auto testing framework:
  - SKILL
  - Antipatterns
  - References / examples